Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 7

ПО КУРСУ

«Алгоритмические языки»

на тему «Рефлексия»

Студент ИУ8-31 (Группа) Р. Д. Хан (И. О. Фамилия)

Преподаватель Р. Д. Исянов (И.О. Фамилия)

2026 г.

Цель работы

Изучить рефлексию C#, создание пользовательских атрибутов и получение метаданных сборки.

Условие задачи

1. Создать библиотеку классов Animal, Cow, Lion, Pig и перечисления по предложенной диаграмме. Реализовать пользовательский атрибут со свойством Comment и применить его к каждому классу.

2. Создать приложение, подключающее библиотеку. Средствами рефлексии сформировать XML-представление диаграммы классов, включая пользовательские атрибуты.

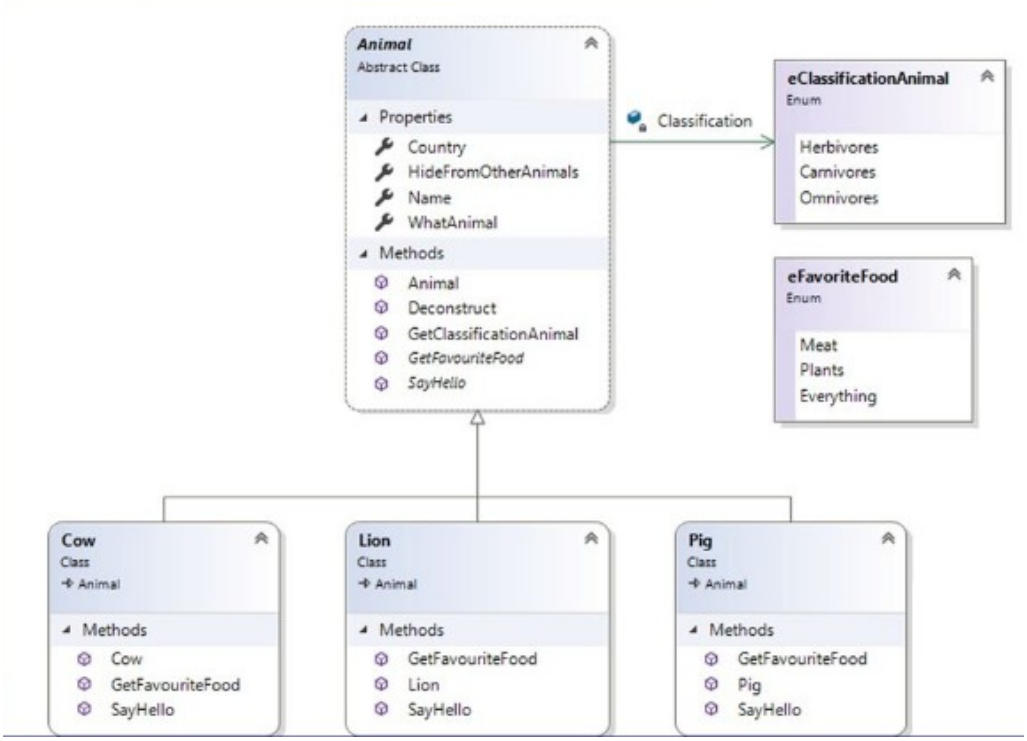


Диаграмма из условия лабораторной работы.

Описание решения

AnimalLibrary — отдельная сборка. Animal является абстрактным классом; Cow, Lion и Pig переопределяют GetFavoriteFood и SayHello. Перечисления задают классификацию и любимую пищу. Типы свойств, не указанные в диаграмме, выбраны по смыслу: строки и bool; связь с классификацией представлена свойством Classification.

typeof(Animal).Assembly получает сборку библиотеки. GetExportedTypes возвращает её публичные типы. Для каждого типа приложение извлекает базовый класс, атрибуты, конструкторы, свойства, поля и методы с параметрами. XDocument сохраняет результат в diagram.xml. Автоматические поля свойств исключены, чтобы не дублировать свойства.

Порядок запуска

Открыть решение Labs07_10.sln. Назначить Lab07 запускаемым проектом. Выполнить запуск; результат — diagram.xml в рабочем каталоге.

Код программы на языке C#

Файл AnimalLibrary/Animal.cs

```
using System.Xml.Serialization;
namespace AnimalLibrary
{
    public enum eClassificationAnimal { Herbivores, Carnivores, Omnivores }
    public enum eFavoriteFood { Meat, Plants, Everything }

    [Comment("Общий абстрактный класс животного")]
    [XmlInclude(typeof(Cow))]
    [XmlInclude(typeof(Lion))]
    [XmlInclude(typeof(Pig))]
    public abstract class Animal
    {
        public string Country { get; set; }
        public bool HideFromOtherAnimals { get; set; }
        public string Name { get; set; }
        public string WhatAnimal { get; set; }
        public eClassificationAnimal Classification { get; set; }

        protected Animal() { }
        protected Animal(string name, string country, bool hide,
            string whatAnimal, eClassificationAnimal classification)
        {
            Name = name;
            Country = country;
            HideFromOtherAnimals = hide;
            WhatAnimal = whatAnimal;
            Classification = classification;
        }
        public void Deconstruct(out string name, out string country)
        {
            name = Name;
            country = Country;
        }
        public eClassificationAnimal GetClassificationAnimal()
        {
            return Classification;
        }
        public abstract eFavoriteFood GetFavoriteFood();
        public abstract string SayHello();
        public override string ToString()
        {
            return Name + ", " + WhatAnimal + ", " + Country
                + ", прячется: " + HideFromOtherAnimals
                + ", " + Classification + ", еда: " + GetFavoriteFood()
                + ", голос: " + SayHello();
        }
    }
}
```

Файл AnimalLibrary/CommentAttribute.cs

```
namespace AnimalLibrary
{
    [AttributeUsage(AttributeTargets.Class, Inherited = false)]
    [Comment("Пользовательский атрибут с комментарием")]
    public class CommentAttribute : Attribute
    {
        public string Comment { get; set; }
        public CommentAttribute(string comment)
        {
            Comment = comment;
```

```
        }
    }
}
```

Файл AnimalLibrary/Cow.cs

```
namespace AnimalLibrary
{
    [Comment("Корова")]
    public class Cow : Animal
    {
        public Cow() { }
        public Cow(string name, string country, bool hide)
            : base(name, country, hide, "Корова",
                eClassificationAnimal.Herbivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Plants;
        }
        public override string SayHello()
        {
            return "My!";
        }
    }
}
```

Файл AnimalLibrary/Lion.cs

```
namespace AnimalLibrary
{
    [Comment("Лев")]
    public class Lion : Animal
    {
        public Lion() { }
        public Lion(string name, string country, bool hide)
            : base(name, country, hide, "Лев",
                eClassificationAnimal.Carnivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Meat;
        }
        public override string SayHello()
        {
            return "P-p-p!";
        }
    }
}
```

Файл AnimalLibrary/Pig.cs

```
namespace AnimalLibrary
{
    [Comment("Свинья")]
    public class Pig : Animal
    {
        public Pig() { }
        public Pig(string name, string country, bool hide)
            : base(name, country, hide, "Свинья",
                eClassificationAnimal.Omnivores) { }
        public override eFavoriteFood GetFavoriteFood()
        {
            return eFavoriteFood.Everything;
        }
        public override string SayHello()
        {
            return "Хрю!";
        }
    }
}
```

Файл Lab07/Program.cs

```
using AnimalLibrary;
using System.Reflection;
using System.Xml.Linq;
namespace Lab07
{
    internal class Program
    {
        static XElement Parameters(ParameterInfo[] parameters)
        {
            XElement result = new XElement("Parameters");
            foreach (ParameterInfo p in parameters)
            {
                result.Add(new XElement("Parameter",
                    new XAttribute("name", p.Name),
                    new XAttribute("type", p.ParameterType.FullName)));
            }
            return result;
        }
        static void Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Assembly assembly = typeof(Animal).Assembly;
            XElement root = new XElement("Assembly",
                new XAttribute("name", assembly.GetName().Name));
            Type[] types = assembly.GetExportedTypes();
            Array.Sort(types, (a, b) => string.CompareOrdinal(a.Name, b.Name));
            foreach (Type type in types)
            {
                XElement node = new XElement("Type",
                    new XAttribute("name", type.FullName),
                    new XAttribute("kind", type.IsEnum ? "enum" : "class"),
                    new XAttribute("abstract", type.IsAbstract));
                if (type.BaseType != null)
                    node.Add(new XAttribute("base", type.BaseType.FullName));
                foreach (CustomAttributeData attribute in type.GetCustomAttributesData())
                {
                    XElement attr = new XElement("Attribute",
                        new XAttribute("type", attribute.AttributeType.FullName));
                    foreach (CustomAttributeTypedArgument arg in attribute.ConstructorArguments)
                        attr.Add(new XElement("Argument", Convert.ToString(arg.Value)));
                    node.Add(attr);
                }
                CommentAttribute comment = type.GetCustomAttribute<CommentAttribute>();
                if (comment != null)
                    node.Add(new XElement("Comment", comment.Comment));
                if (type.IsEnum)
                {
                    foreach (string value in Enum.GetNames(type))
                        node.Add(new XElement("Value", value));
                }
                BindingFlags flags = BindingFlags.Public | BindingFlags.NonPublic
                    | BindingFlags.Instance | BindingFlags.Static | BindingFlags.DeclaredOnly;
                foreach (ConstructorInfo c in type.GetConstructors(flags))
                    node.Add(new XElement("Constructor",
                        new XAttribute("access", c.IsPublic ? "public" : "protected/private"),
                        Parameters(c.GetParameters())));
                foreach (PropertyInfo p in type.GetProperties(flags))
                    node.Add(new XElement("Property", new XAttribute("name", p.Name),
                        new XAttribute("type", p.PropertyType.FullName),
                        new XAttribute("read", p.CanRead), new XAttribute("write", p.CanWrite)));
                foreach (FieldInfo f in type.GetFields(flags))
                {
                    if
(f.IsDefined(typeof(System.Runtime.CompilerServices.CompilerGeneratedAttribute)))
                        continue;
                    node.Add(new XElement("Field", new XAttribute("name", f.Name),
                        new XAttribute("type", f.FieldType.FullName)));
                }
                foreach (MethodInfo m in type.GetMethods(flags))
```

```
                {
                    if (m.IsSpecialName) continue;
                    node.Add(new XElement("Method", new XAttribute("name", m.Name),
                        new XAttribute("return", m.ReturnType.FullName),
                        new XAttribute("abstract", m.IsAbstract),
                        new XAttribute("virtual", m.IsVirtual), Parameters(m.GetParameters())));
                }
                root.Add(node);
                Console.WriteLine(type.Name + (comment == null ? "" : ": " + comment.Comment));
            }
            string path = args.Length > 0 ? args[0] : "diagram.xml";
            new XDocument(root).Save(path);
            Console.WriteLine("XML сохранён: " + Path.GetFullPath(path));
        }
    }
}
```

Результаты выполнения программы

Скомпилированы библиотека и приложение. XML успешно создан и повторно разобран. Проверены 7 типов, абстрактность Animal, наследование и наличие Comment у Animal, Cow, Lion и Pig. Ниже приведён фактический вывод запуска.

Lab07_run.txt

```
Animal: Общий абстрактный класс животного
CommentAttribute: Пользовательский атрибут с комментарием
Cow: Корова
Lion: Лев
Pig: Свинья
eClassificationAnimal
eFavoriteFood
XML сохранён: /workspace/scratch/6823d4720338/compile_work/diagram.xml
```

Фрагмент созданного XML

```
<?xml version="1.0" encoding="utf-8"?>
<Assembly name="AnimalLibrary">
  <Type name="AnimalLibrary.Animal" kind="class" abstract="true" base="System.Object">
    <Attribute type="AnimalLibrary.CommentAttribute">
      <Argument>Общий абстрактный класс животного</Argument>
    </Attribute>
    <Attribute type="System.Xml.Serialization.XmlIncludeAttribute">
      <Argument>AnimalLibrary.Cow</Argument>
    </Attribute>
    <Attribute type="System.Xml.Serialization.XmlIncludeAttribute">
      <Argument>AnimalLibrary.Lion</Argument>
    </Attribute>
    <Attribute type="System.Xml.Serialization.XmlIncludeAttribute">
      <Argument>AnimalLibrary.Pig</Argument>
    </Attribute>
    <Comment>Общий абстрактный класс животного</Comment>
    <Constructor access="protected/private">
      <Parameters />
```

Выводы по работе

Создана библиотека классов по диаграмме. Пользовательские атрибуты и структура типов получены рефлексией и записаны в XML. При добавлении нового публичного типа в библиотеку приложение обнаружит его без изменения списка типов в коде.